TALLER DE PYTHON & IA APLICADA AL NEGOCIO · SECTOR BANCA

Python Aplicado al Análisis de Negocio

Sesión 1

FACILITADOR

Marlon Leandro
Software Archiect | AI Engineer
MBA / Ing. Sistemas © 2026

FECHA

21 de abril de 2026

Bienvenida y Contexto

Perfil del Participante

Profesionales de banca: Finanzas, Riesgos, Operaciones, Comercial, Auditoría y Canales Digitales.

Cero Código Previo

No se requiere experiencia en programación. Aprovecharemos tu conocimiento de negocio, KPIs y Excel.

Metodología Práctica

Balance de 70% práctica y 30% teoría. Usamos Google Colab — sin instalar nada.

Enfoque de Negocio

Contenido orientado a toma de decisiones y quick wins. Casos reales del sector financiero.

Objetivos de la Sesión 1

Al finalizar esta sesión de 3 horas, el participante logrará:

- ✓ Entender por qué Python es clave estratégica en la banca actual.
- ✓ Revisar casos reales: automatización de reportes, análisis de clientes y detección de anomalías.
- ✓ Aprender los fundamentos prácticos de Python orientados exclusivamente a negocio.
- ✓ Analizar datos con Pandas: cargar, filtrar, calcular KPIs y agrupar información bancaria.
- 💻 Realizar un mini caso práctico interactivo utilizando datos simulados de banca.

¿Por qué Python en el Negocio?

El valor ejecutivo de adoptar Python para profesionales financieros:

- ⚡ **Eficiencia:** Reduce drásticamente las horas manuales en reportes y conciliaciones.
- ✅ **Calidad:** Minimiza errores humanos, garantizando mayor trazabilidad y repetibilidad.
- 🚀 **Velocidad:** Acelera la generación de insights de días a tan solo minutos.
- 📈 **Escalabilidad:** Procesa desde miles hasta millones de registros sin cambiar la lógica subyacente.
- 🔗 **Integración:** Se conecta de forma nativa con Excel/CSV, bases de datos y APIs corporativas.
- 🤖 **Tendencia:** Es la tecnología fundacional para implementar IA y automatización en banca.

Python en Banca: Panorama y Adopción

Tendencias Clave

📊 **Autoservicio Analítico** Reducción de dependencia de TI para extraer y analizar datos.

📋 **Exigencia Regulatoria** Mayor trazabilidad, calidad y reproducibilidad en reportes.

👤 **Analítica Cliente 360°** Segmentación avanzada y personalización en tiempo real.

🛡️ **Monitoreo de Riesgos** Detección de anomalías y fraudes con mayor velocidad y precisión.

Adopción en la Práctica

Áreas Principales

Finanzas, Riesgos, Prevención de Fraude, Comercial y Operaciones.

Casos Típicos

Scoring simple, alertas transaccionales y automatización de reporting regulatorio.

Enfoque Recomendado

Empezar identificando "**quick wins**" de negocio y luego escalar las soluciones.

CASO DE USO #1

Automatización de Reportes

1 — Orígenes
Core bancario + Excel/CSV de distintas áreas

›

2 — Limpieza
Estandarizar formatos, fechas, monedas y códigos

›

3 — Cálculo KPIs
Morosidad (NPL), ROE por segmento, volúmenes

›

4 — Generación
Creación de reportes automáticos (Excel/PDF)

›

5 — Distribución
Envío por correo o carga en carpeta compartida

- ⏱️ 50-70% menos tiempo en elaboración manual
- ✅ Consistencia total y menor riesgo de error humano
- 🔍 Auditoría y trazabilidad 100% reproducibles

Más Casos de Uso en Banca

Análisis de Clientes

🎯 Segmentación Avanzada
Clasificación por valor (RFM), Customer Lifetime Value (CLV) y propensión a compra.

🔔 Identificación de Churn
Detección temprana de fugas y diseño de ofertas de retención (next-best-action).

🛒 Cross y Upselling
Recomendación de productos basada en comportamiento transaccional histórico.

Detección de Anomalías

⚡ Alertas Transaccionales
Identificación de montos, frecuencias o ubicaciones inusuales en tiempo real.

📉 Detección de Outliers
Análisis de patrones atípicos de gasto, ingresos y operaciones en canales digitales.

🛡️ Prevención de Riesgos
Señales tempranas de riesgo operativo, crediticio y fraude financiero.

COMPARATIVA

Excel vs Python

Excel

📊 Análisis Ad-hoc
Ideal para exploraciones rápidas y tableros simples.

⚡ Familiar y Rápido
Interfaz conocida; muy potente con tablas dinámicas.

⚠️ Limitaciones Técnicas
Problemas con grandes volúmenes, repetibilidad y auditoría.

Usar Excel cuando: datasets pequeños/medianos y análisis puntual.

Python

🤖 Automatización Robusta
Automatiza procesos repetitivos y maneja millones de registros.

🔄 Reproducible y Escalable
Código auditable integrado con bases de datos y APIs.

🧠 Habilita Modelos Avanzados
Base para segmentación, scoring de riesgo y detección de anomalías.

Usar Python cuando: tareas recurrentes, múltiples fuentes o se requiera trazabilidad.

El Problema que Resuelve Python

Puntos de dolor comunes en las áreas de negocio:

- 📋 Tareas manuales y repetitivas (copiar/pegar, limpiar datos en Excel).
- ⚠️ Errores humanos y múltiples versiones de un mismo reporte.
- ⏳ Demoras para generar KPIs clave y realizar conciliaciones.
- 🔗 Dificultad para cruzar fuentes de datos (Core + CRM + Excel).
- 📉 Procesos lentos, poco escalables y sin trazabilidad real.
- 💻 Alta dependencia del área de TI para cambios simples o extracciones.

Python convierte estos procesos frágiles en flujos de trabajo robustos y automáticos.

Variables y Tipos de Datos

¿Qué es una Variable? Una "caja" con nombre donde guardamos información temporal para usarla después.

Tipos Comunes: Números (enteros/decimales), Texto, Fechas y Booleanos (verdadero/falso).

Ejemplos Bancarios:
saldo = 15000.50 · segmento = "Pyme" · es_moroso = False

Buenas Prácticas: Usar nombres claros como **tasa_interes_anual**. Mantener unidades consistentes.

¿Para qué sirve? Crear filtros automáticos, calcular KPIs masivos y aplicar reglas sistemáticas.

```python
# Tipos de datos básicos
amount = 12500.75      # float: monto crédito
rate = 0.145           # float: tasa 14.5%
customer_name = "Ana Perez"
product_code = "CC-PLAT"
days_past_due = 5      # int: días de mora
is_past_due = days_past_due > 0

# Fechas (requiere librería)
from datetime import datetime
payment_date = datetime(2026, 4, 15)

# Cálculo rápido (KPI simple)
monthly_interest = amount * rate / 12
print(f"Interés mensual: S/ {monthly_interest:.2f}")
```

Estructuras de Control: if/else y loops

```python
# Clasificación de mora
if days_past_due > 30:
    status = "Mora (>30 días)"
elif days_past_due > 0:
    status = "Atraso leve (1-30 días)"
else:
    status = "Al día"

# Procesamiento de transacciones
flags = []
count_pos = 0
for tx in txs:
    if tx["amount"] ≥ 10000:
        flags.append({
            "id": tx["cust_id"],
            "alert": "HIGH_VALUE"
        })
    if tx["channel"] == "POS":
        count_pos += 1

print(f"Total TX en POS: {count_pos}")
```

¿Cuándo usarlas en Banca?

if/elif/else: reglas de negocio, validaciones de crédito, clasificación de riesgo.

loops for/while: iterar listas de clientes o lotes de transacciones.

Buenas Prácticas

Evitar loops anidados muy profundos.
Para miles o millones de registros, usar Pandas — vectorizado y mucho más rápido.

Listas y Diccionarios: Clientes y Transacciones

```python
# Listas de diccionarios (Clientes)
customers = [
    {"id": 101, "name": "Ana", "segment": "Pyme"},
    {"id": 102, "name": "Luis", "segment": "Retail"}
]

# Listas de diccionarios (Transacciones)
txs = [
    {"cust_id": 101, "amount": 250.0, "channel": "POS"},
    {"cust_id": 102, "amount": 1200.0, "channel": "App"}
]

# Acceso a datos específicos
first_name = customers[0]["name"]   # "Ana"
tx_amount  = txs[1]["amount"]       # 1200.0

# List comprehension (filtro rápido)
small_txs = [t["amount"] for t in txs
             if t["amount"] < 500]
```

Conceptos Clave

Listas [ ]: colecciones ordenadas. Ideales para agrupar múltiples registros.

Diccionarios { }: clave-valor. Cada fila de Excel es un diccionario.

Usos Prácticos en Banca

Construir la "**Vista 360**" del cliente. Preparar listas segmentadas para campañas. Filtrar transacciones sospechosas con **list comprehensions**.

HERRAMIENTA PRINCIPAL

Introducción a Pandas: Análisis de Datos

La herramienta clave para transformar datos en insights de negocio:

- ✓ 📊 Qué es: Librería de Python para manejar tablas de datos (como Excel, pero programable y automatizable).
- ✓ 🔧 Qué permite: Leer Excel/CSV, unir múltiples fuentes, filtrar registros, agrupar datos y calcular KPIs masivos.
- ✓ 🏦 Ejemplos de negocio: Calcular morosidad por segmento, ROE por producto, o ticket promedio por sucursal en segundos.
- ✓ ⚡ Beneficios directos: Extrema rapidez, trazabilidad total de los pasos y reproducibilidad (mismo resultado cada vez).
- ✓ ☁️ Entorno de trabajo: Todo se ejecutará en Google Colab, directamente en la nube y sin necesidad de instalación.

Flujo de Trabajo Completo con Datos

1 — Cargar
Desde Excel/CSV (CRM, core, riesgos)

›

2 — Limpiar
Estandarizar fechas, monedas y códigos

›

3 — Filtrar
Clientes activos, períodos, segmentos

›

4 — Analizar
KPIs: NPL, ROE por segmento, ticket

›

5 — Reportar
Excel/PDF, envío programado

- Tiempo: de horas a minutos
- Calidad: menos errores, más trazabilidad
- Repetibilidad: mismo resultado cada vez

Caso Integrador: Análisis de Cartera

```python
# 1. Cargar datos de Excel
import pandas as pd
cust = pd.read_excel("clientes.xlsx")
loans = pd.read_excel("prestamos.xlsx",
        parse_dates=["due_date"])

# 2. Filtrar clientes activos
cust_active = cust.loc[
    cust["status"] == "Active",
    ["id", "segment"]]

# 3. Calcular KPIs a nivel cliente
ref_date = pd.Timestamp("2026-04-20")
loans["dpd"] = (ref_date - loans["due_date"])\
             .dt.days.clip(lower=0)
kpis = (loans.groupby("customer_id")
        .agg(balance=("balance","sum"),
             dpd_max=("dpd","max"))
        .reset_index())
kpis["is_past_due"] = kpis["dpd_max"] > 0
```

Objetivo

Flujo completo: carga, filtros, cruce y cálculo de KPIs para obtener insights de negocio accionables.

Paso a Paso

1. Cargar archivos Excel de base.
2. Filtrar solo cartera activa.
3. Calcular mora y consolidar KPIs.
4. Unir datos y resumir por segmento.
5. Extraer top portafolios y alertas.

Resultados Esperados

Visión clara de cartera por segmento.
Detección temprana de mora alta (>8%).
Script automatizable para reporte diario.

Cierre y Próximos Pasos

¡El viaje hacia la automatización y el análisis avanzado apenas comienza!

Quick Wins

→ Automatizar reporte de cartera semanal desde Excel.

→ Limpieza y conciliación rápida de 2 fuentes (CRM + Core).

→ Generar alertas simples por montos inusuales o pagos vencidos.

Buenas Prácticas

→ Empezar simple: una tarea repetitiva a la vez.

→ Usar Google Colab: evita configuraciones complejas.

→ Nombres de variables claros y fechas estándar (YYYY-MM-DD).

→ Validar resultados con el área de negocio.

→ Próximos Pasos

→ Sesión 2: traer un dataset anonimizado de tu área.

→ Practicar filtros y agrupaciones básicas en Pandas.

→ Prepárate para el caso integrador y la aplicación de IA.

El objetivo no es ser programadores — es tomar mejores decisiones de negocio usando tecnología.

¿Preguntas?

Marlon Leandro
Software Archiect | AI Engineer
MBA / Ing. Sistemas © 2026